



CRYPTOGRAPHIC PROTOCOLS AND TLS 1.3

Tom Chothia

Introduction

- A summary of the 2nd material on cryptographic protocols
 - NS protocol
 - Station-to-station protocol
 - Diffie Hellman
 - Forward security.
 - Basic TLS
- New Material:
 - Full TLS 1.2
 - TLS 1.3
 - Key Compromise Impersonation attacks.

Redirected traffic to UK Atomic Weapons Establishment



Protocol Goals

- Keep the **keys set up secret**.

The Needham-Schroeder Public Key Protocol

Assume Alice and Bob know each others public keys, can they set up a symmetric key?

1. $A \rightarrow B : E_B(N_a, A)$
2. $B \rightarrow A : E_A(N_a, N_b)$
3. $A \rightarrow B : E_B(N_b)$

$E_x(_)$ means public key encryption

N_a and N_b can then be used to generate a symmetric key

An Attack Against the Needham-Schroeder Protocol

The attacker acts as a man-in-the-middle:

1. $A \rightarrow C : E_C(N_a, A)$

1'. $C(A) \rightarrow B : E_B(N_a, A)$

2'. $B \rightarrow C(A) : E_A(N_a, N_b)$

2. $C \rightarrow A : E_A(N_a, N_b)$

3. $A \rightarrow C : E_C(N_b)$

3'. $C(A) \rightarrow B : E_B(N_b)$

The Corrected Version

A very simple fix:

1. $A \rightarrow B : E_B(N_a, A)$
2. $B \rightarrow A : E_A(N_a, N_b, B)$
3. $A \rightarrow B : E_B(N_b)$

Protocol Goals

- Keys set up are **secret and fresh**.
- **Authenticate: both parties**, i.e., both are talking to who they really believe they are talking to.

Forward Secrecy

$A \rightarrow B : E_B(N_a, A)$

$B \rightarrow A : E_A(N_a, N_b, B)$

$A \rightarrow B : E_B(N_b)$

$B \rightarrow A : \{M\}_{key(Na, Nb)}$

Secure against the
“standard” attacker.

- intercept, replay,
delete, alter.

- But what about the governments?
- After the protocol runs, governments can legally force people to hand over their private keys.
- Can we protect against this?

Forward Secrecy

- A protocol has “Forward Secrecy” if it keeps the message secret from an attacker who has:
 - A recording of the protocol run
 - The long-term keys of the principals.
- Protection against
 - a government that can force people to give up their keys,
 - or hackers that might steal them.

Protocol Goals

- Keys set up are **secret and fresh**.
- **Authenticate: both parties**, i.e., both are talking to who they really believe they are talking to.
- **Forward Secrecy**.

Diffie-Hellman

- Diffie-Hellman is a widely used key agreement protocol.
- It relies on some number theory:
 - $a \bmod b = n$ where for some “m” : $a = m.b + n$
- The protocol uses two public parameters
 - generator “g” (often 160 bits long)
 - prime “p” (often 1024 bits long)

Diffie-Hellman

- Alice and Bob pick random numbers r_A and r_B and find “ $t_A = g^{r_A} \bmod p$ ” and “ $t_B = g^{r_B} \bmod p$ ”
- The protocol just exchanges these numbers:
 1. $A \rightarrow B : t_A$
 2. $B \rightarrow A : t_B$
- “Alice” calculates “ $t_B^{r_A} \bmod p$ ” and “Bob” “ $t_A^{r_B} \bmod p$ ” this is the key:
 - $K = g^{r_A r_B} \bmod p$

Diffie-Hellman

$$1. A \rightarrow B : \overbrace{(g \times g \times \dots \times g)}^{r^A \text{ times}} \bmod p$$

$$2. B \rightarrow A : \overbrace{(g \times g \times \dots \times g)}^{r^B \text{ times}} \bmod p$$

Alice calculates

$$\begin{aligned} & \overbrace{\overbrace{(g \times \dots \times g)}^{r^B \text{ times}} \times \overbrace{(g \times \dots \times g)}^{r^B \text{ times}} \times \dots \times \overbrace{(g \times \dots \times g)}^{r^B \text{ times}}}^{r^A \text{ times}} \bmod p \\ &= \overbrace{(g \times g \times g \times \dots \times g)}^{r^B r^A \text{ times}} \bmod p \end{aligned}$$

Diffie-Hellman

$$1. A \rightarrow B : \overbrace{(g \times g \times \dots \times g)}^{r^A \text{ times}} \bmod p$$

$$2. B \rightarrow A : \overbrace{(g \times g \times \dots \times g)}^{r^B \text{ times}} \bmod p$$

Bob calculates

$$\begin{aligned} & \overbrace{\overbrace{(g \times \dots \times g)}^{r^A \text{ times}} \bmod p \times \overbrace{(g \times \dots \times g)}^{r^A \text{ times}} \bmod p \times \dots \times \overbrace{(g \times \dots \times g)}^{r^A \text{ times}} \bmod p}^{r^B \text{ times}} \\ &= \overbrace{(g \times g \times g \dots \times g)}^{r^A r^B \text{ times}} \bmod p \end{aligned}$$

Station-to-Station Protocol

1. $A \rightarrow B : g^x$
2. $B \rightarrow A : g^y, E_A(S_B(g^y, g^x))$
3. $A \rightarrow B : E_B(S_A(g^x, g^y))$
4. $B \rightarrow A : \{ M \}_{g^{xy}}$

$S_X(_)$ means
signed by X

- x, y, g^{xy} are not stored after the protocol run.
- A & B's keys don't let attacker read M
- STS has forward secrecy

A Protocol without Mutual Belief

1. $A \rightarrow B : g^x$
 - 1'. $B(A) \rightarrow C : g^x$
 - 2'. $C \rightarrow B(A) : g^y, E_A(S_C(g^y, g^x))$
2. $B \rightarrow A : g^y, E_A(S_B(g^y, g^x))$
3. $A \rightarrow B : E_B(S_A(g^y, g^x))$
 - 3'. $B(A) \rightarrow C : E_C(S_A(g^x, g^y))$
4. $A \rightarrow C : \{M\}_{g^{xy}}$

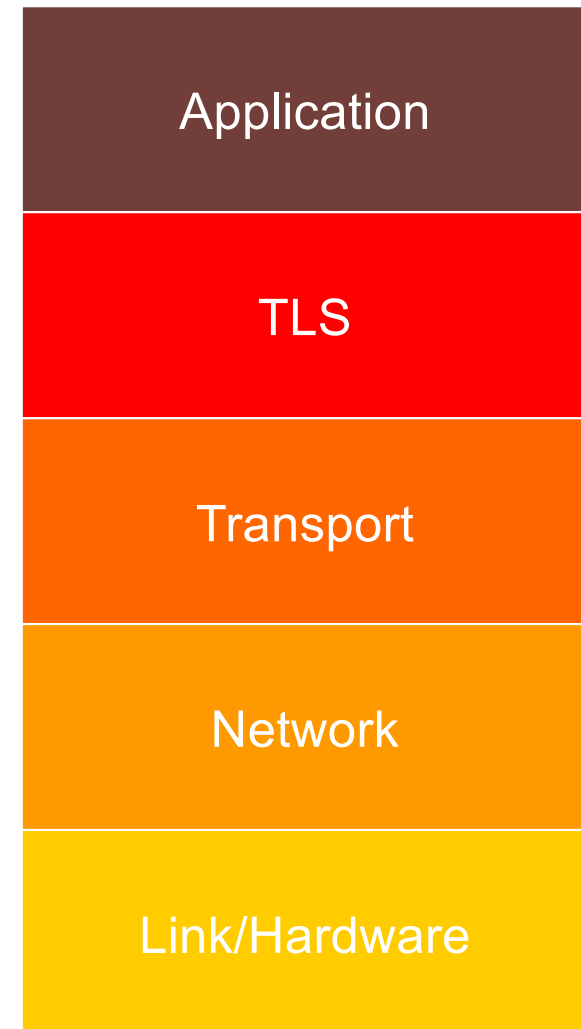
Alice thinks she is talking to Bob but is really talking to Charlie.

The Internet Protocol Stack with TLS

The TLS layer runs between the Application and Transport layer.

The encryption is transparent to the Application layer.

Normal TCP and IP protocols etc. can be used at the low layers



Transport Layer Security TLS 1.2

1. $C \rightarrow S : N_C$
2. $S \rightarrow C : N_S, \text{Cert}_S$
3. $C \rightarrow S : E_S(K_{\text{seed}}), \{\text{Hash1}\}_{K_{CS}}$
4. $S \rightarrow C : \{\text{Hash2}\}_{K_{CS}}$

Hash 1 = $\#(N_C, N_S, E_S(K_{\text{seed}}))$

Hash 2 = $\#(N_C, N_S, E_S(K_{\text{seed}}), \{\text{Hash1}\}_{K_{CS}})$

K_{CS} is a session key based key derivation function (kdf) on
 N_C, N_S & K_{seed} ,

All data is then encrypted with K_{CS} and hashed for integrity.

Transport Layer Security (TLS)

1. $C \rightarrow S : N_C, \text{Possible CipherSuites}$
2. $S \rightarrow C : N_S, \text{Cert}_S, \text{CipherSuite}$
3. $C \rightarrow S : E_S(K_seed), \{\text{Hash1}\}_{K_{CS}}$
4. $S \rightarrow C : \{\text{Hash2}\}_{K_{CS}}$

Hash 1 = $\#(N_C, N_S, E_S(K_seed))$

Hash 2 = $\#(N_C, N_S, E_S(K_seed), \{\text{Hash1}\}_{K_{CS}})$

K_{CS} is a session key based key derivation function (kdf) on
 N_C, N_S & K_seed ,

Transport Layer Security (TLS)

1. $C \rightarrow S$: **ClientHello**: N_C , Possible CipherSuites
2. $S \rightarrow C$: **ServerHello**: N_S , CipherSuite
2. $S \rightarrow C$: **Certificate**: $Cert_S$
2. $S \rightarrow C$: **ServerHelloDone**
3. $C \rightarrow S$: **ClientKeyExchange** $E_S(K_seed)$,
3. $C \rightarrow S$: **ChangeCipherSpec**
3. $C \rightarrow S$: {Hash of previous message} $_{K_{CS}}$
4. $S \rightarrow C$: {Hash2} $_{K_{CS}}$

K_{CS} is a session key based key derivation function (kdf) on
 N_C , N_S & K_seed ,

Transport Layer Security (TLS)

1. $C \rightarrow S$: **ClientHello**: N_C , Possible CipherSuites
2. $S \rightarrow C$: **ServerHello**: N_S , CipherSuite
2. $S \rightarrow C$: **Certificate**: $Cert_S$
2. $S \rightarrow C$: **ServerHelloDone**
3. $C \rightarrow S$: **ClientKeyExchange**: $E_S(K_{seed})$
3. $C \rightarrow S$: **ChangeCipherSpec**
3. $C \rightarrow S$: {Hash of previous message} $_{K_{CS}}$
4. $S \rightarrow C$: **ChangeCipherSpec**
4. $S \rightarrow C$: {Hash of previous message} $_{K_{CS}}$

TLS-DHE

A variant uses Diffie-Hellman for “forward secrecy”

i.e., if someone gets the servers key later, they can't go back and break a recording of the traffic.

1. $C \rightarrow S : N_C$, Possible CipherSuites
2. $S \rightarrow C : N_S$, g^x , Cert_S , $\text{Sign}_S(\#(N_C, N_S, g^x))$, CipherSuite
3. $C \rightarrow S : g^y$, $\{\#(\text{All previous messages})\}_{\text{kdf}(N_C, N_S, g^{xy})}$
4. $S \rightarrow C : \{\#(\text{All previous messages})\}_{\text{kdf}(N_C, N_S, g^{xy})}$

Transport Layer Security (TLS)

1. $C \rightarrow S$: **ClientHello**: N_C , Possible CipherSuites
2. $S \rightarrow C$: **ServerHello**: N_S , Cert_S , CipherSuite
2. $S \rightarrow C$: **Certificate**: Cert_S
2. $S \rightarrow C$: **ServerKeyExchange**: g^x , $\text{Sign}(\#(N_C, N_S, g^x))$
2. $S \rightarrow C$: **ServerHelloDone**
3. $C \rightarrow S$: **ClientKeyExchange**: g^y
3. $C \rightarrow S$: **ChangeCipherSpec**
3. $C \rightarrow S$: {Hash of previous message} $\text{kdf}(N_C, N_S, g^{xy})$
4. $S \rightarrow C$: **ChangeCipherSpec**
4. $S \rightarrow C$: {Hash of previous message} $\text{kdf}(N_C, N_S, g^{xy})$

Client

Server

ClientHello: N_C , Possible CipherSuites

ServerHello: N_S , $Cert_S$, CipherSuite

Certificate: $Cert_S$

ServerKeyExchange: g^x , $Sign(\#(N_C, N_S, g^x))$

ServerHelloDone

ClientKeyExchange: g^y

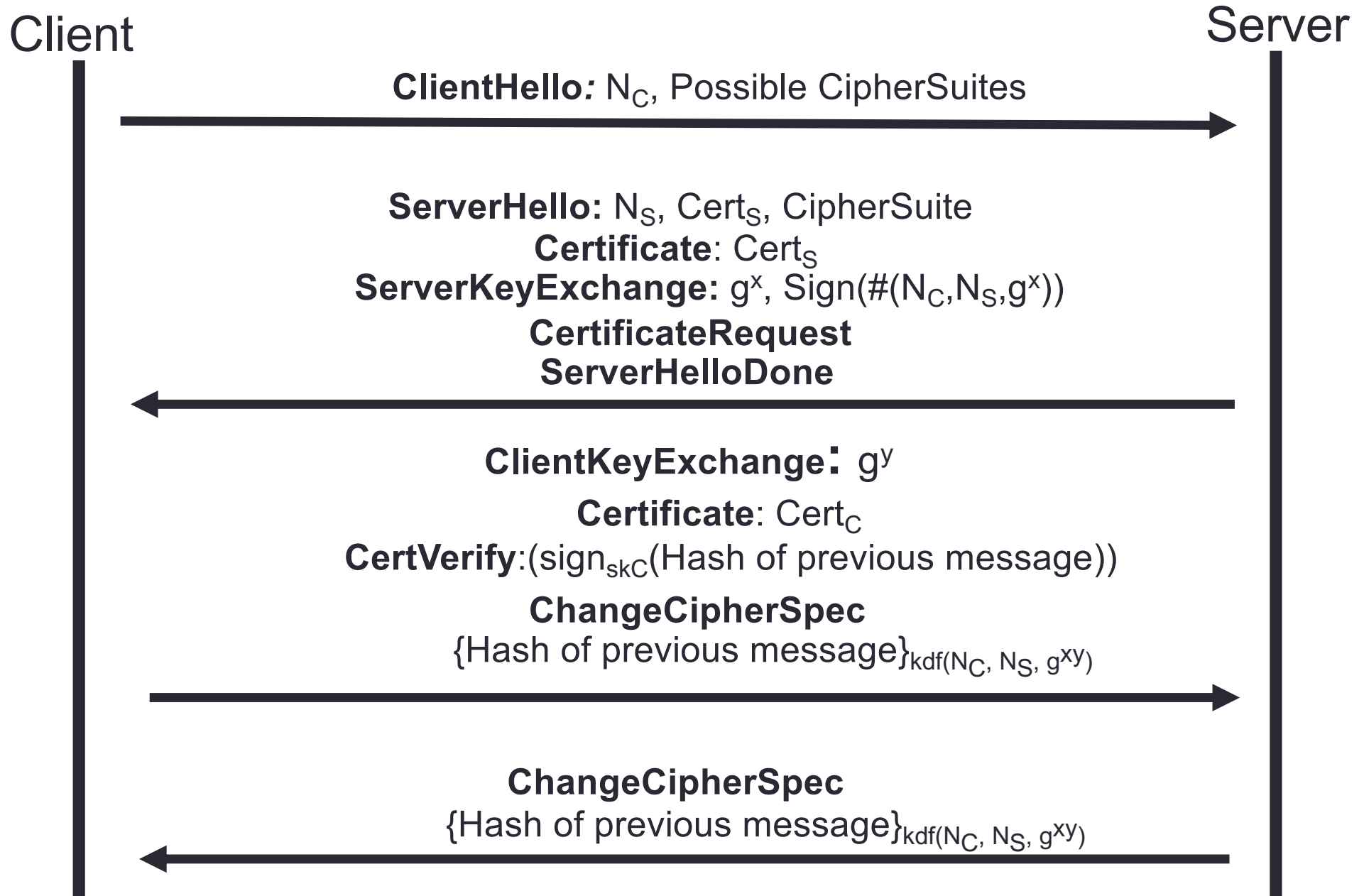
ChangeCipherSpec

$\{\text{Hash of previous message}\}_{kdf(N_C, N_S, g^{xy})}$

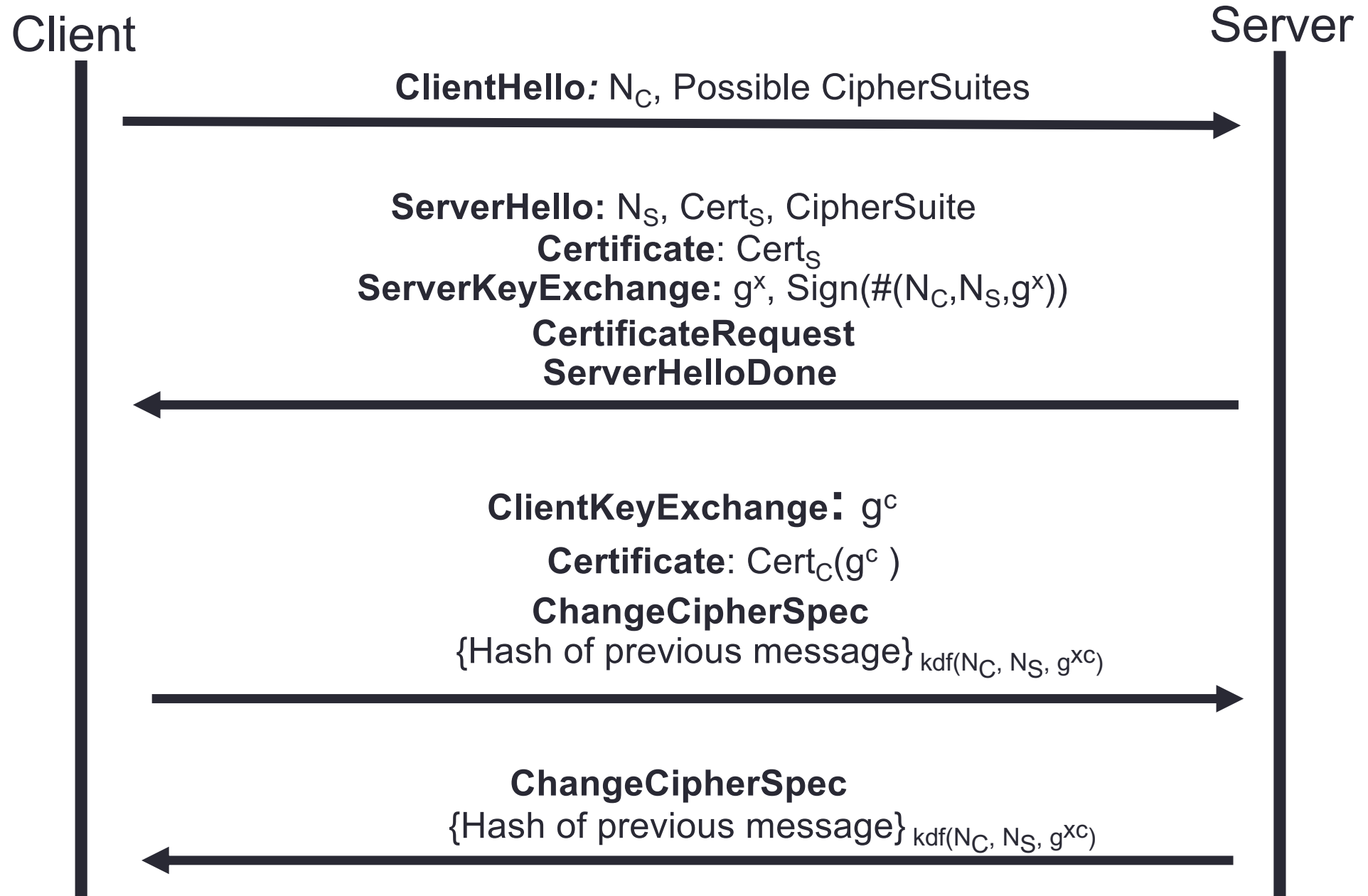
ChangeCipherSpec

$\{\text{Hash of previous message}\}_{kdf(N_C, N_S, g^{xy})}$

TLS 1.2 with Client Certificate



TLS 1.2 with Static Client Certificate



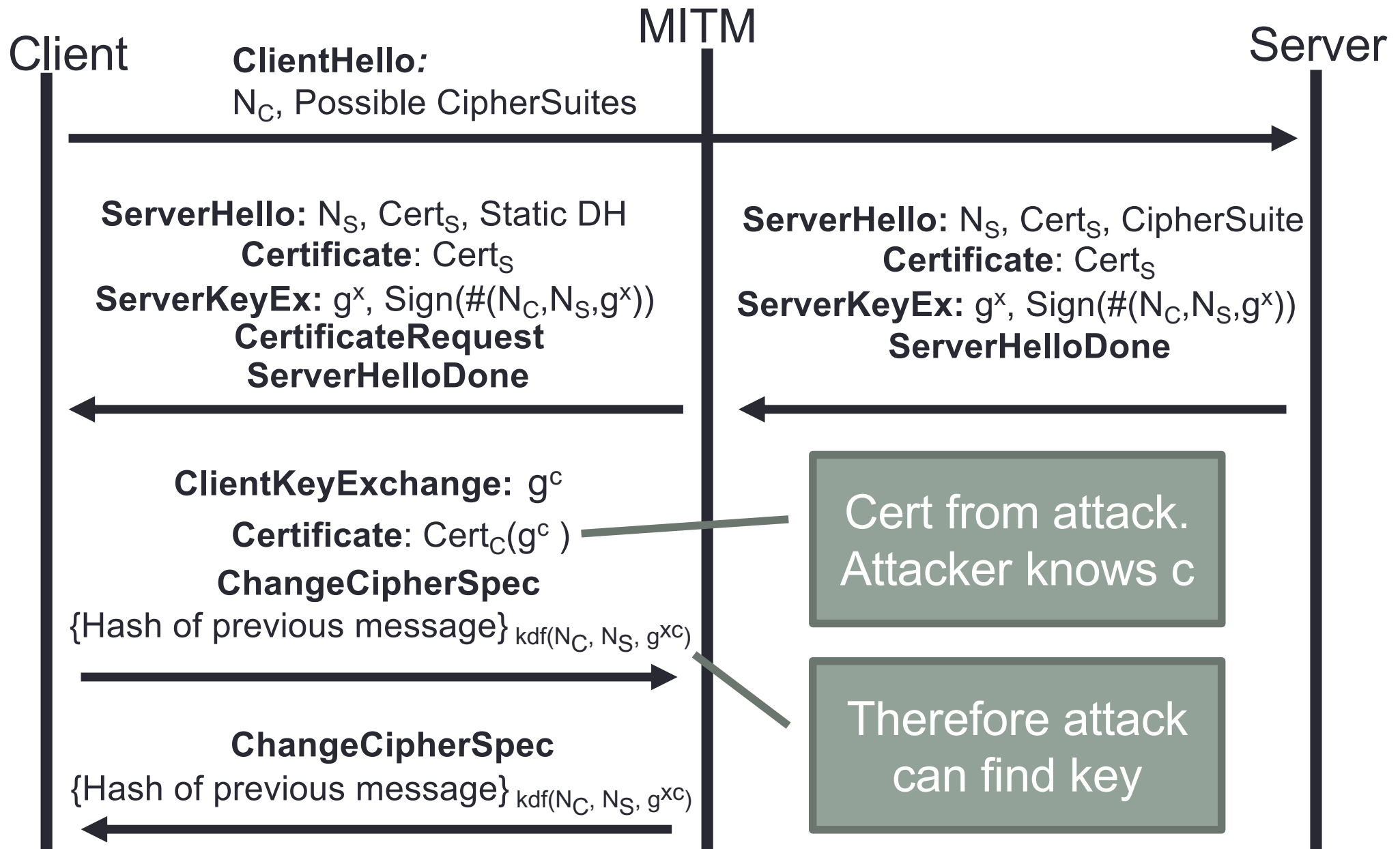
TLS 1.2 is a mess

- There are too many cipher suites.
- There are too many options.
- Due to this there have been lots of attacks against particular cipher suites, or rare options.
 - E.g. https://en.wikipedia.org/wiki/Transport_Layer_Security_Security
- You need 2 round trips before you can start sending data.
- Many attacks based on mistakes in implementations.
 - The complexity makes this more likely.
- TLS 1.3 fixes this.

Key Compromise Impersonation Attack

- In a KCI attack: attack can control client's key.
- The attack is successful if the attack can pretend to be any server.
- E.g. your employer says you must install a client certificate to connect to your place of work and then uses this to MITM any connection.
- This is an edge case attack: if attack can install a certificate in the trust store then TLS is trivially broken.
- TLS with static DH and Client Certificate is vulnerable to a KCI attack.

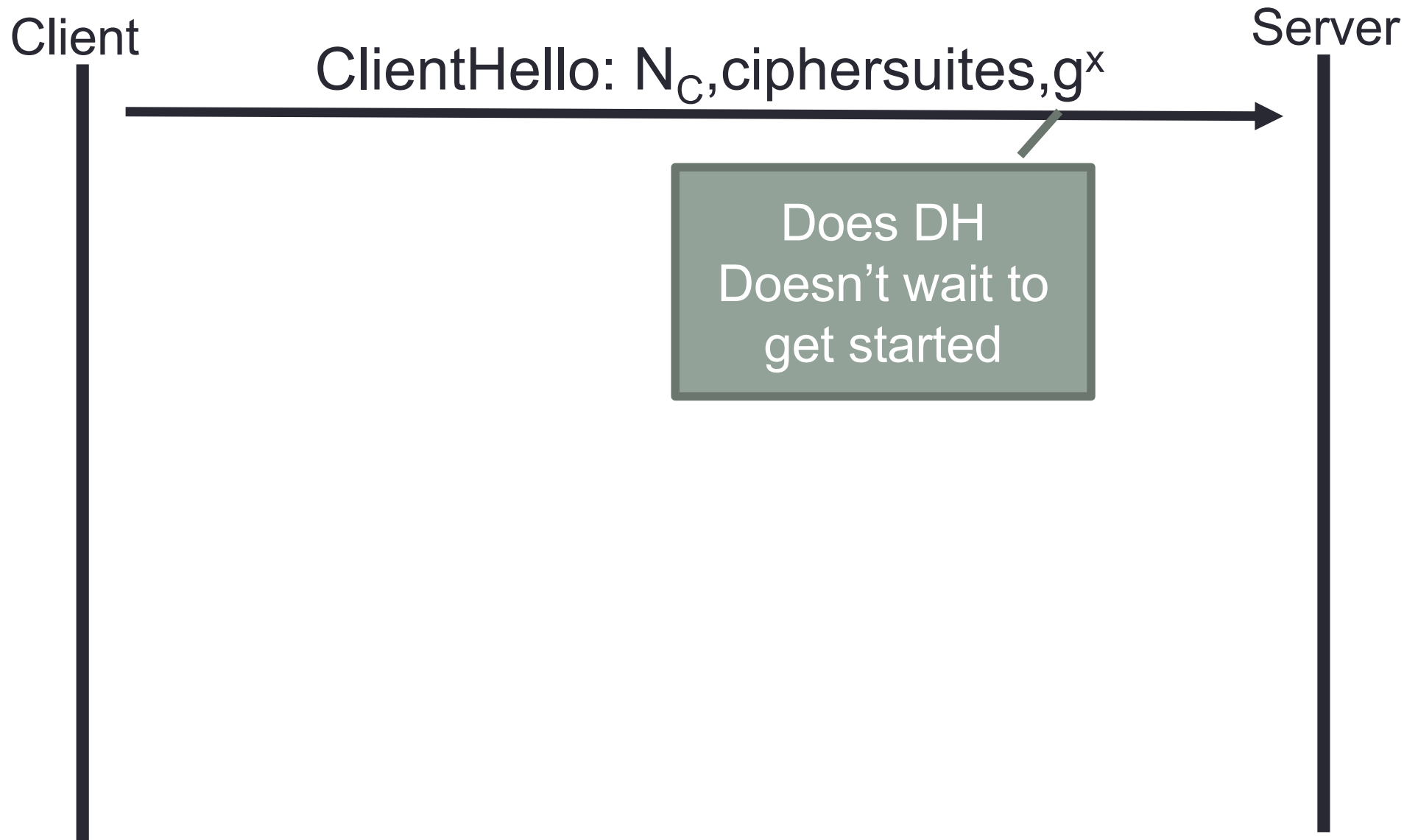
TLS 1.2 KCI vs Static Client Certificate



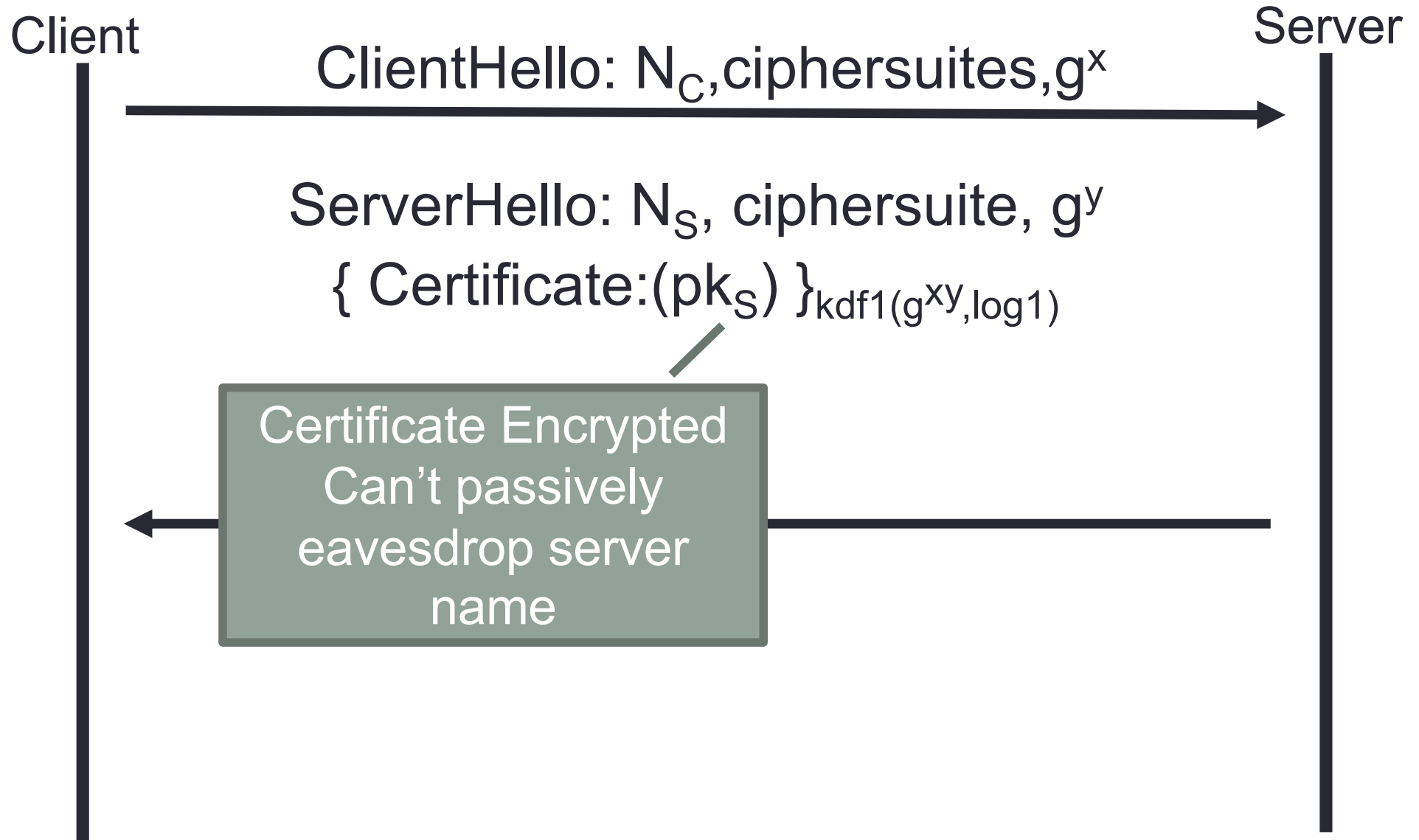
Protocol Goals

- Keys set up are **secret and fresh**.
- **Authenticate: both parties**, i.e., both are talking to who they really believe they are talking to.
- **Forward Secrecy**.
- **No weak cipher**.
- **Key Compromise Impersonation Attack**.
- **Protocol should not be complex or have lots of different modes**.

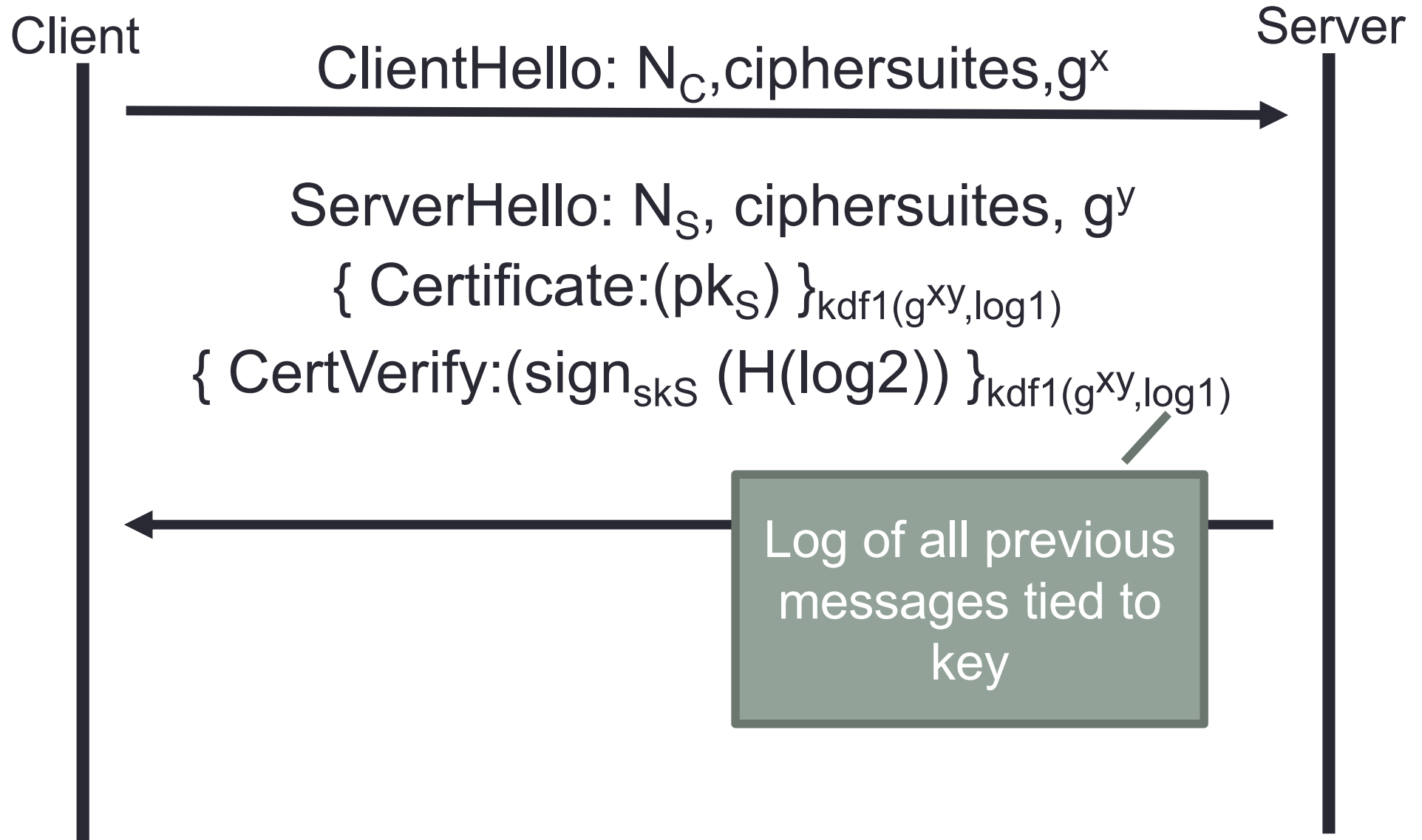
Core TLS 1.3



Core TLS 1.3



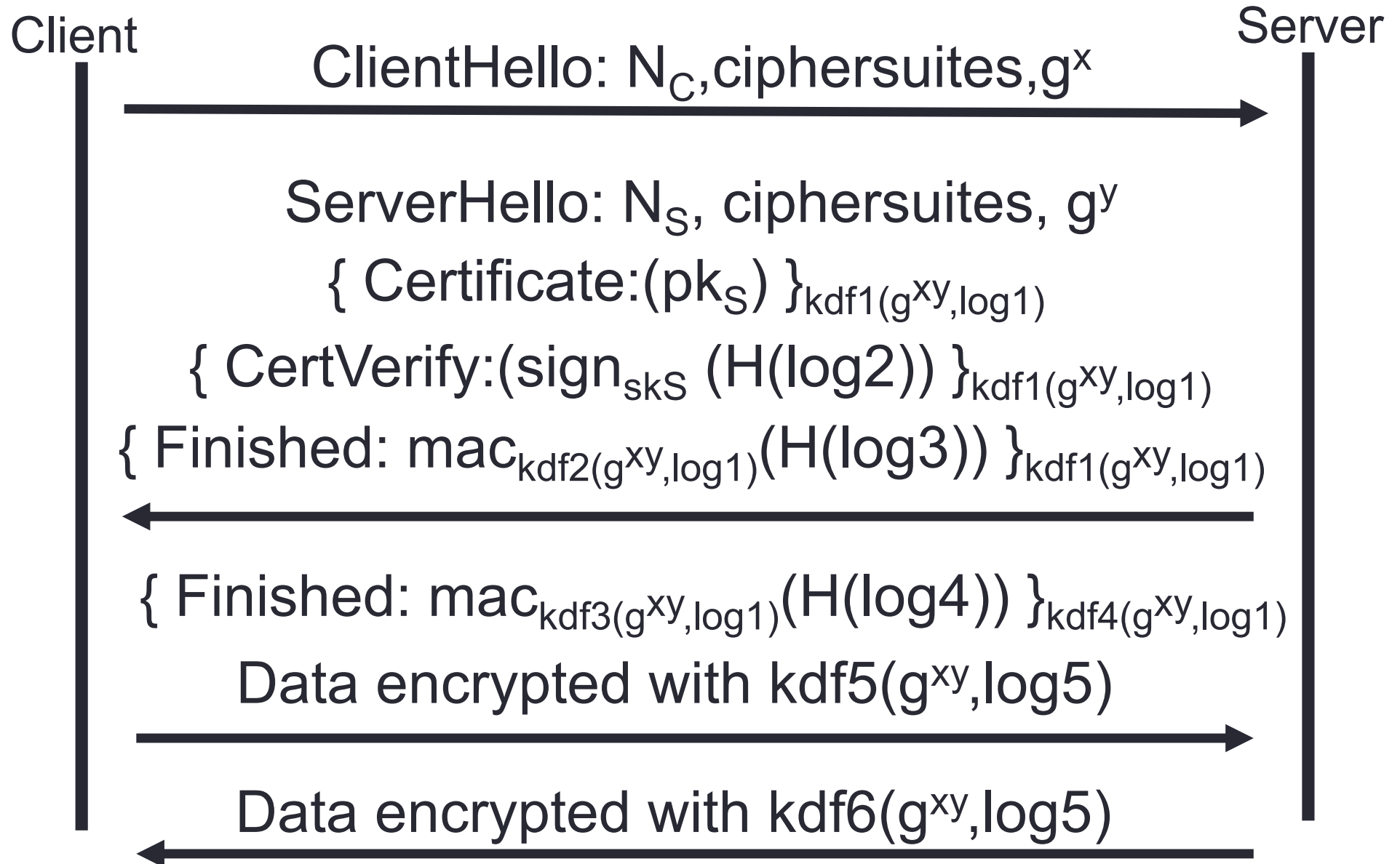
Core TLS 1.3



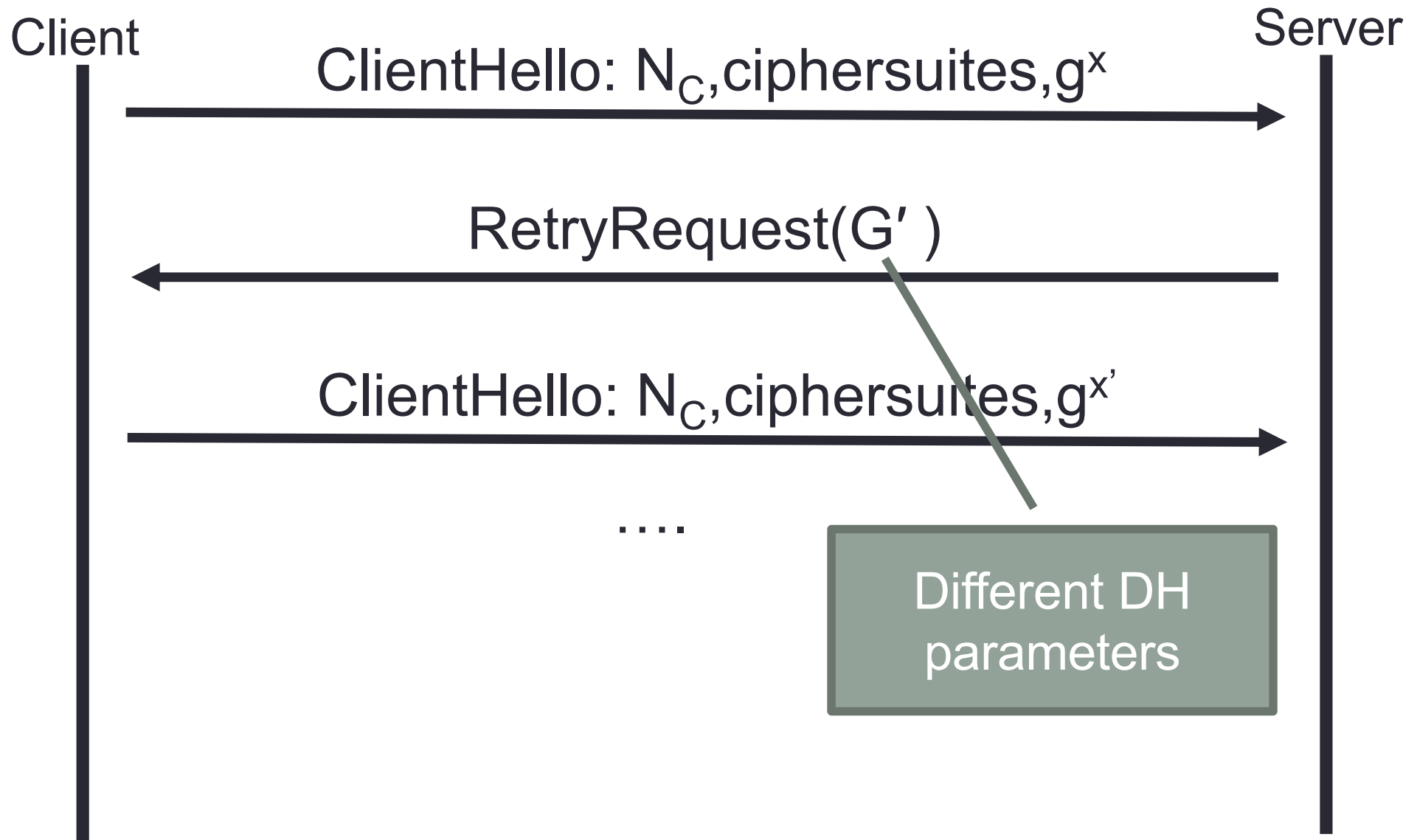
Core TLS 1.3



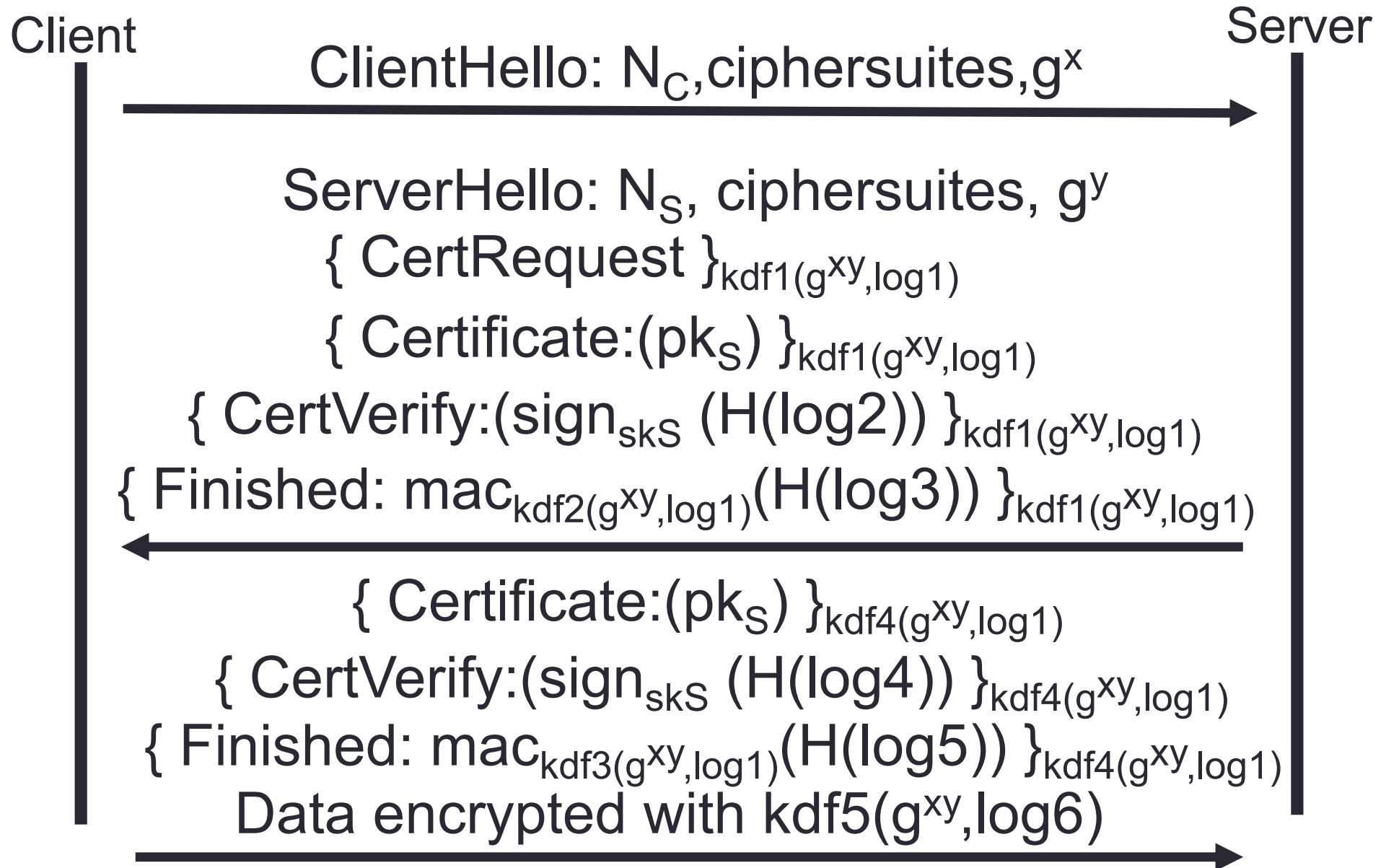
Core TLS 1.3



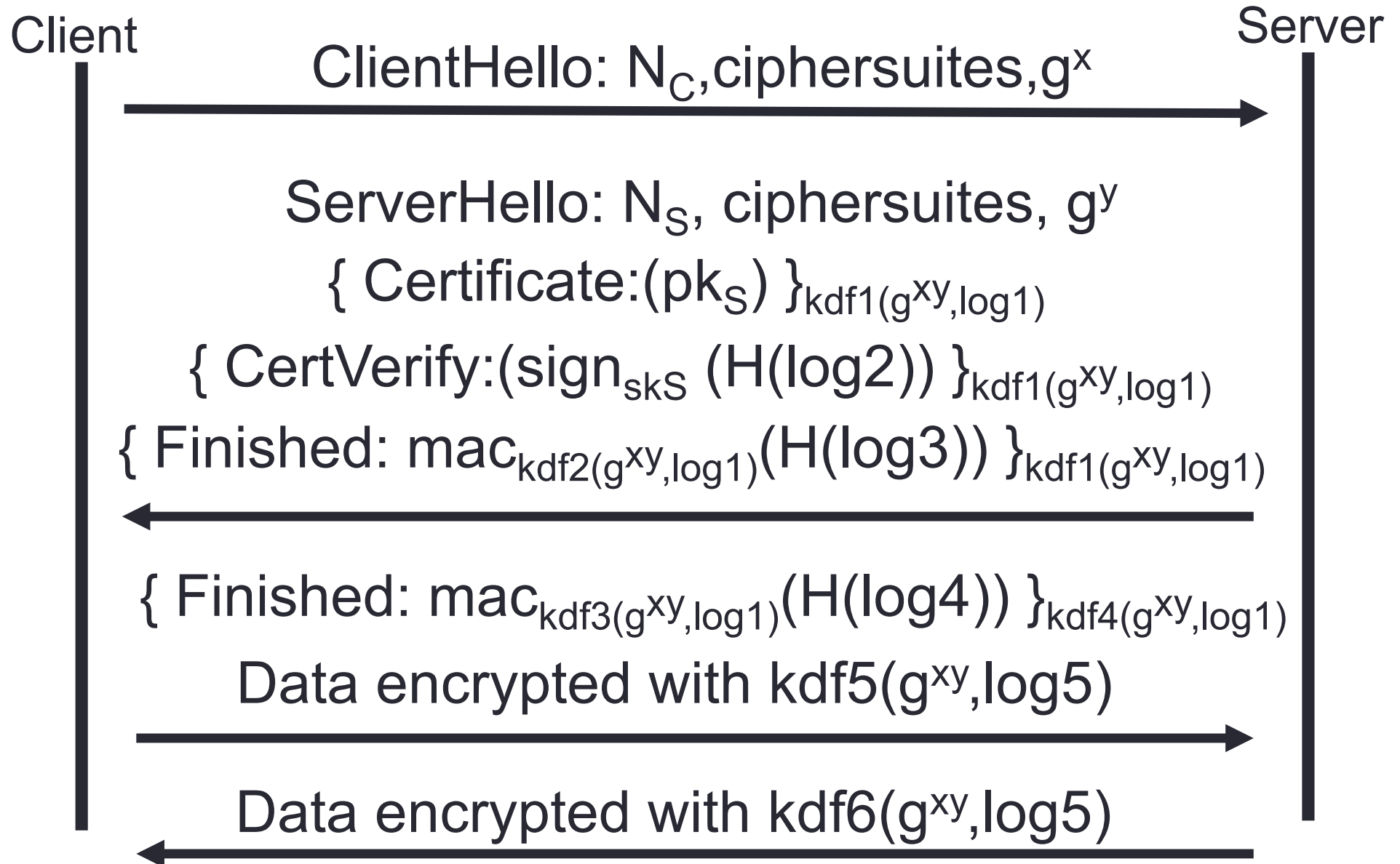
TLS 1.3: Server requests different DH



TLS 1.3: Client Certificate



Core TLS 1.3



TLS 1.3 improvements over TLS 1.2

- Only 5 very secure cipher suites:
 - TLS_AES_256_GCM_SHA384
 - TLS_CHACHA20_POLY1305_SHA256
 - TLS_AES_128_GCM_SHA256
 - TLS_AES_128_CCM_8_SHA256
 - TLS_AES_128_CCM_SHA256
- All Elliptic Curve Diffie Hellman, for forward secure (see next week).
- Few mode of operations (e.g. RSA, no static DH)
- Hides the server name from passive evesdroppers.
- Starts sending data after 1 round trip of messages.

Sources

- The official TLS 1.3 spec (RFC):
<https://tools.ietf.org/html/rfc8446>
- The list of TLS past attacks:
https://en.wikipedia.org/wiki/Transport_Layer_Security_Security
- The paper on KCI attack vs TLS 1.2:
<https://www.usenix.org/conference/woot15/workshop-program/presentation/hlauschek>
- A good paper on formal verification of TLS 1.3:
<https://www.ieee-security.org/TC/SP2017/papers/234.pdf>

Summary

- A summary of the 2nd material on cryptographic protocols
 - NS protocol
 - Diffie Hellman
 - Station-to-station protocol
 - Forward security.
 - Basic TLS
- New Material:
 - Full TLS 1.2
 - TLS 1.3
 - Key Compromise Impersonation attacks.
- Next Lecture: An in depth look at the EMV cryptographic protocols used in bank cards.